



A novel attribute-based searchable encryption scheme with attribute revocation

Zixin Xiong¹ · Jun Ye¹

Received: 6 November 2025 / Accepted: 6 September 2026
© The Author(s) 2026

Abstract

In this paper, a searchable attribute-based encryption scheme with attribute revocation is proposed to solve the problem of secure data sharing and efficient retrieval in cloud storage. Based on the bilinear pairings construction, the scheme includes five algorithms: system initialization, key generation, data encryption, search and decryption, and attribute revocation, which realizes fine-grained access control and keyword ciphertext search. Through the attribute update key mechanism, the key and ciphertext can be updated without re-encryption when the user attribute changes. Security analysis proves that the scheme satisfies adaptive security in the standard model, and performance evaluation shows that the efficiency of attribute revocation is superior to the existing schemes, especially suitable for large-scale cloud storage environment.

Keywords Big data security · Attribute-based encryption · Searchable encryption · Privacy security · Cloud security

1 Introduction

In recent years, the rapid advancement of cloud computing has driven a growing number of individuals and organizations to outsource their massive volumes of sensitive data to cloud servers. This trend is primarily motivated by the cloud's inherent advantages, including enhanced scalability, flexible resource allocation, and improved cost-efficiency [1, 2]. Nevertheless, this data outsourcing paradigm introduces profound concerns regarding data privacy and security. Given that cloud service providers (CSPs) are typically considered semi-trusted entities, sensitive data must be encrypted prior to outsourcing to mitigate risks of unauthorized access and potential data breaches [3].

To facilitate controlled data sharing while preserving data confidentiality, Attribute-Based Encryption (ABE) has emerged as a foundational cryptographic primitive [4, 5]. Among various ABE variants, Ciphertext-Policy ABE (CP-ABE) is particularly well-adapted to cloud storage scenarios. In CP-ABE, data owners (DOs) encrypt data under a flexible

access policy defined over a set of attributes; only data users (DUs) whose attribute sets satisfy the predefined policy can successfully decrypt the ciphertext. This mechanism enables fine-grained access control, making it a core technology for secure cloud-based data sharing systems [6].

However, with the exponential growth of encrypted data in the cloud, a critical challenge emerges: how to efficiently locate specific target files within a large-scale encrypted dataset without first decrypting the entire corpus. To address this issue, Searchable Encryption (SE) was proposed [7], which allows DUs to perform keyword-based searches over encrypted data. Building on this, the concept of Attribute-Based Keyword Search (ABKS) [8, 9] further integrates ABE with SE, enabling keyword search functionality that is inherently tied to access control. Specifically, a DU can only generate a valid search trapdoor if their attribute set meets the required policy. Importantly, the CSP can execute the search operation without gaining any knowledge of the underlying data or the search query, thereby preserving data and query privacy.

Despite the notable advantages of existing ABKS schemes, they often fail to address a critical requirement for practical deployment: efficient user attribute revocation. In real-world scenarios, a user's attributes may change due to factors such as role promotions, department transfers, or security breaches [10]. In such cases, revoking the user's access rights to future encrypted data becomes essential to maintain system secu-

✉ Jun Ye
yejun@hainanu.edu.cn

Zixin Xiong
xiongzixin@hainanu.edu.cn

¹ School of Cyberspace Security, Hainan University, Haikou, China

ity. Conventional naive solutions—such as re-encrypting all data associated with the revoked attribute or re-issuing keys to all non-revoked users—incur prohibitive computational and communication overheads, especially in large-scale cloud systems [11]. Consequently, designing an ABKS scheme that supports efficient and immediate attribute revocation remains an unresolved and challenging research problem.

Existing efforts to integrate revocation mechanisms into ABE or ABKS systems suffer from several limitations. For instance, some schemes delegate revocation management to the CSP, which exacerbates trust issues. Other schemes [12] rely on heavyweight cryptographic tools like proxy re-encryption, which introduces significant performance bottlenecks. A truly practical solution must minimize trust assumptions on the CSP while ensuring that the revocation process is lightweight, scalable, and compatible with large-scale cloud deployments.

To address the aforementioned challenges, this paper proposes a novel and efficient Searchable Attribute-Based Encryption scheme that supports immediate attribute revocation (referred to as Revocable ABKS, R-ABKS for brevity). The key contributions of our work are summarized as follows:

- **Unified Fine-Grained Access Control and Keyword Search:** We design a CP-ABE-based framework that seamlessly integrates secure, privacy-preserving keyword search over encrypted data. DOs can define complex access policies, and only users with attribute sets satisfying these policies can generate valid search trapdoors and decrypt the corresponding ciphertexts.
- **Lightweight and Immediate Attribute Revocation:** We introduce a dedicated revocation mechanism that avoids the overhead of full data re-encryption or mass key re-issuance. When an attribute is revoked, only the ciphertext components associated with the revoked attribute and the key components of non-revoked users need to be updated. This ensures the revocation process is efficient, immediate, and does not require the DO to download, decrypt, or re-encrypt the entire dataset.

The core innovation of R-ABKS lies in the introduction of an attribute revocation mechanism that does not require re-encryption. By updating the attribute key, the key and ciphertext can be locally updated, significantly reducing the revocation cost. This mechanism is combined with ciphertext keyword search and supports practical cloud storage scenarios with multiple data owners, multiple users, and multiple keywords, achieving a dynamic balance between fine-grained access control and efficient retrieval. This solution satisfies adaptive security in the standard model and is particularly suitable for large-scale cloud environments.

1.1 Related work

The convergence of Attribute-Based Encryption (ABE) and Searchable Encryption (SE) has emerged as a promising paradigm for secure and fine-grained data management in cloud environments. Our work builds upon and extends this line of research. This section reviews the relevant literature in two key areas: Searchable ABE and attribute revocation mechanisms.

• Searchable attribute-based encryption

The foundational goal of ABE [13, 14] is to enable one-to-many encryption where access is defined by a user's attributes. To address the need for searching over encrypted data, Li et al. [15] pioneered Searchable Encryption, which allowed for keyword search on symmetrically encrypted data. The fusion of these concepts led to Searchable ABE (ABKS), which permits a data user to search over data encrypted under an access policy without decrypting it first.

Early ABKS schemes [16, 17] supported single-keyword search. However, to meet practical demands, subsequent research focused on multi-keyword search. Schemes like EMKV-ABSE [18] and MKSVABE [19] achieved this by embedding multiple keywords into the ciphertext and trapdoor, allowing for more expressive queries. Our proposed scheme falls into this category, supporting conjunctive multi-keyword search to enhance query flexibility.

Another critical challenge in ABKS is resisting Keyword Guessing Attacks (KGA), where an adversarial cloud server attempts to deduce the keywords in a trapdoor by generating and testing trapdoors for guessed keywords. Several countermeasures have been proposed, such as designing schemes with an outside tester [20] or leveraging a secure enclave, as explored in our work, to blind the trapdoor and break the direct link between the server's guesses and the actual index.

In 2023, Bera S et al. [24] proposed a searchable attribute-based encryption scheme named MediCare, which is used for the secure storage and retrieval of electronic medical records (EMR) in the cloud environment. This scheme was the first to simultaneously achieve fine-grained access control, anonymous authentication of EMR owners, support for keyword search using Boolean formulas, keyword privacy protection, constant decryption cost for users, and non-interactive verification of search results. In 2024, Bera S et al. [25] addressed the issues of limited search functionality, low efficiency, and weak security of the existing ABPRE-KS scheme, and proposed a new scheme ABPRE-BKS that supports Boolean keyword search. This scheme achieves efficient multi-keyword combination search in large attribute

domains, with constant decryption computational load at the user end, and can resist various types of selected ciphertext/keyword attacks.

- Attribute revocation in ABE systems

Attribute revocation is a cornerstone of dynamic access control but is notoriously difficult to achieve efficiently in cryptographic systems. In ABE, when a user's attribute is revoked, all ciphertexts encrypted under policies containing that attribute should become inaccessible to the user without requiring the data owner to re-encrypt all affected data.

Existing revocation mechanisms can be broadly classified into two categories. The first is direct revocation, where the data owner explicitly specifies a revocation list during encryption. While straightforward, this approach shifts a significant burden onto the data owner, who must be aware of the latest revocation status, making it less scalable.

The second, and more prevalent, category is indirect revocation, which typically relies on a trusted third party (e.g., a Key Generation Center) to periodically update user keys and/or ciphertexts. Schemes like [21] introduce a semi-trusted proxy that can re-encrypt ciphertexts using update keys, rendering them unreadable to revoked users. However, many early ABKS schemes, including EMKV-ABSE [18] and MKSVABE [19], lack an efficient revocation mechanism altogether. A few schemes, such as ABMKSE [20], have integrated revocation, but often at a high computational cost or with limitations on functionality. Fan et al. [26] proposed a scheme named rABEKS, which is an attribute-based encryption scheme that supports multi-keyword search and efficient user revocation. This scheme achieves fine-grained access control and ciphertext keyword search. The user key and ciphertext length are fixed and independent of the number of attributes, and effective user permission revocation is achieved through the revocation list.

Our work directly addresses this gap. We propose a novel and efficient indirect revocation mechanism that seamlessly integrates with our ABKS framework. Our scheme achieves immediate and secure attribute revocation without imposing a heavy computational load on the data owner, a distinct advantage over many existing schemes, as highlighted in the functional comparison.

In summary, while significant progress has been made in ABKS, a scheme that simultaneously supports efficient multi-keyword search, robust privacy protection against KGA, and practical attribute revocation remains elusive. Our work is novel in that it holistically integrates these three critical features into a unified and efficient framework, overcoming the limitations of prior art and advancing the state-of-the-art for practical, secure cloud data sharing.

2 Preliminaries

2.1 Notation

The main notations used in this paper are listed in Table 1.

2.2 Bilinear maps

A bilinear map, defined as $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_T$ over groups of prime order p , must satisfy three key properties: *bilinearity* (i.e., $e(g^a, h^b) = e(g, h)^{ab}$ for all $g, h \in \mathbb{G}_1, a, b \in \mathbb{Z}_p$), *non-degeneracy* (i.e., $e(g, g)$ generates $\mathbb{G}_T$ if g generates $\mathbb{G}_1$), and *computational efficiency*. This structure, typically instantiated via pairings on elliptic curves, serves as the foundation for numerous advanced cryptographic protocols.

2.3 Access structures

Let $U = \{u_1, u_2, \dots, u_n\}$ be a set of attributes (e.g., “department=R&D”). $f S \in \mathbb{A}$ (user attributes satisfy the policy) and $S \subseteq T \subseteq U$, then $T \in \mathbb{A}$ (adding attributes preserves authorization).

LSSS represents $\mathbb{A}$ via:

- A *share matrix* $M \in \mathbb{Z}_p^{l \times n}$ (l rows, n attributes) and a *row function* $\rho : \{1, \dots, l\} \rightarrow U$ (maps rows to attributes);
- A secret $s \in \mathbb{Z}_p^*$ is split into shares $\lambda_i = \mathbf{v} \cdot M_i$ (dot product of random vector $\mathbf{v} = (s, v_2, \dots, v_n)$ and M 's i -th row M_i);

Table 1 Notation and parameter descriptions

Notation	Description
λ	Security parameter
G, G_T, p, g, e	Bilinear groups, prime order, generator, pairing
H_0, H_1	Cryptographic hash functions
U	Global attribute set
η_v, PAK_v	Attribute secret exponent and public key
a, α, β	Master secret parameters
MSK, MPK	Master secret and public keys
S_{ID}	User attribute set
t, r	Random exponents for key generation
K_1, K_2, K_3, K_i	User secret key components
k_θ, F_θ	AES key and data file
$\mathbb{A} = (M, \rho)$	Access policy (matrix and mapping)
$\vec{v}, \xi_i$	Secret vector and shares
γ_i, ε_i	Random exponents for encryption
$h(x)$	Revocation polynomial
W, I_W	Keyword set and index
$\text{Td}_{W'}, \varphi$	Search trapdoor and random exponent
$\text{AUK}_{v'}, \text{RL}_{v'}$	Attribute update key and revocation list

- Authorized sets reconstruct s via linear combinations of shares; unauthorized sets cannot.

3 System and security model of R-ABKS

3.1 System model

As shown in Fig. 1, The responsibilities of the four entities are defined as follows.

Trusted Third Authority (TTA) manages system initialization and key generation. It executes Algorithm 1 to create public parameters, master keys and private keys for users based on their attributes via Algorithm 2.

Cloud Service Provider (CSP) stores encrypted data and indexes. It processes search requests by running the DataSearch algorithm from Algorithm 4, verifying user-submitted trapdoors against stored indexes.

Data Owner (DO) builds searchable indexes and encrypts data. Using Algorithm 3, it encrypts files under access policy and generates a corresponding keyword index before uploading both to the CSP.

Data User (DU) searches and decrypts data. It creates search trapdoors using its private key (Algorithm 4) and can decrypt authorized ciphertexts locally to recover the original files.

3.2 Formal definition

This cryptographic scheme formally defines a Searchable Attribute-Based Encryption system with revocation capabilities. The following algorithms outline the core operations for secure data management and controlled access in a cloud environment.

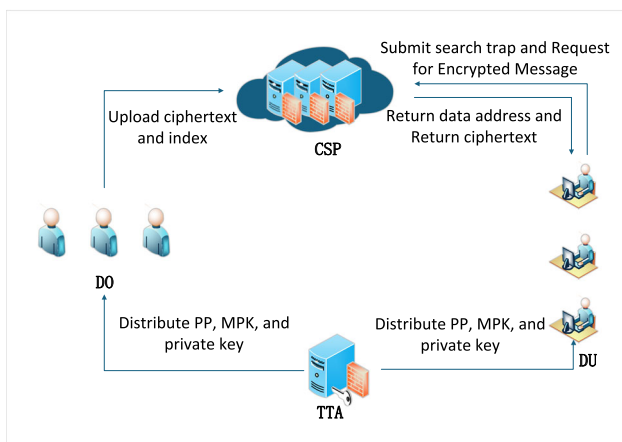


Fig. 1 System model

1. System initialization

Setup (λ) $\rightarrow$ (PP, MPK, MSK): It is executed by the TTA, which enters security parameter λ and outputs public parameters.

2. User key generation

KeyGen (MSK, S_{ID}) $\rightarrow$ ($sk_{du}, (pk_{do}, sk_{do})$): It is the generation algorithm of private key performed by the TTA which enters MSK , and user attribute set S_{ID} . It outputs DU private key sk_{du} and DO public-private key pair (pk_{do}, sk_{do}) .

3. Data encryption

Encrypt ($MPK, F, k_{\theta}, \mathbb{A}$) $\rightarrow$ (CT, I_W): It is executed by the DO through taking MPK , file set $F = \{F_{\theta}\}$, an encapsulated key k_{θ} , and access structure $\mathbb{A}$ as input, then generates ciphertext CT and keyword index I_W .

4. Search and decryption

TrapdoorGen (MPK, sk_{du}, W') $\rightarrow Td_{W'}$: It is the trapdoor generation algorithm performed by the DU which enters MPK , sk_{du} , and keyword set W' , and outputs search trapdoor $Td_{W'}$.

DataSearch ($I_W, Td_{W'}, \mathbb{A}$) $\rightarrow add$: It is the search algorithm executed by the CSP which enters I_W , $Td_{W'}$, and $\mathbb{A}$, and outputs data address add if the search is successful.

Decrypt (CT, sk_{du}) $\rightarrow k_{\theta}$: It run by the DU which takes CT and sk_{du} as input, and outputs the symmetric key k_{θ} .

5. Attribute revocation

Revoke ($v', RL_{v'}$) $\rightarrow (sk'_{du}, CT', I'_W)$: It is the revocation algorithm executed by the TTA which enters revoked attribute v' and revocation list $RL_{v'}$, and outputs updated secret key sk'_{du} , updated ciphertext CT' , and updated index I'_W .

3.3 Threat model

In the proposed construction, DO honestly executes the protocol, and forwards ciphertext and search index to CSP. It is considered that CSP is honest but curious, which will honestly execute the search protocol but may analyze encrypted data and search patterns to infer sensitive information. Regarding the adversary $\mathcal{A}$, it is presumed to be probabilistic polynomial-time (PPT) and can adaptively query user secret keys for non-satisfying attribute sets. CSP will not, however, conspire with the revoked users. Furthermore, AA is assumed to be honest, which is accountable for establishing system and creating user private keys. In addition, we also assume that the adversary $\mathcal{A}$ can perform chosen keyword attacks and attribute value guessing attacks, but cannot win the security games with non-negligible advantage under the q -parallel BDHE and CDH assumptions.

3.4 Security definitions

Definition 1 (IND-sCP-CPA security) Let Π be an R-ABKS, and $\mathcal{A}$ be a probabilistic polynomial-time adversary. The IND-sCP-CPA (the indistinguishability-based selective ciphertext-policy attack on the plaintext) security is defined through the following game:

1. *Initialization*: $\mathcal{A}$ selects a challenge access structure $\mathbb{A}^*$ and sends it to $\mathcal{C}$.
2. *Setup*: $\mathcal{C}$ runs $\text{Setup}(\lambda)$ to get (MPK, MSK) and provides MPK to $\mathcal{A}$.
3. *Query phase 1*: $\mathcal{A}$ may adaptively query:
 - $\mathcal{O}_{\text{KeyGen}}(\text{ID}, S_{\text{ID}})$: For S_{ID} not satisfying $\mathbb{A}^*$.
 - $\mathcal{O}_{\text{AUK}}(v')$: For any $v' \in U$.
 - $\mathcal{O}_{\text{DOKeyGen}}()$: Data owner key generation.
 - $\mathcal{O}_{\text{Trapdoor}}(\text{sk}_{\text{du}}, W')$: For W' not containing challenge keywords.
4. *Challenge*: $\mathcal{A}$ submits k_0, k_1 ; $\mathcal{C}$ encrypts k_σ under $\mathbb{A}^*$ to get CT^* and sends it.
5. *Query Phase 2*: $\mathcal{A}$ continues queries under same restrictions.
6. *Guess*: $\mathcal{A}$ outputs σ' .

The advantage is defined as:

$$\text{Adv}_{\mathcal{A}}^{\text{IND-sCP-CPA}}(\lambda) = \left| \Pr[\sigma' = \sigma] - \frac{1}{2} \right|$$

Π is secure if this advantage is negligible.

Definition 2 (IND-KGA-AVGA Security) Let Π be an R-ABKS, and $\mathcal{A}$ be a probabilistic polynomial-time adversary. Consider the following game for the IND-KGA-AVGA security, which is to evaluate whether the scheme can resist indistinguishability under keyword guessing and attribute value guessing attacks:

1. *Initialization*: $\mathcal{A}$ chooses a challenge keyword set W^* . $\mathcal{C}$ runs $\text{Setup}(\lambda)$, gets (MPK, MSK), and sends MPK to $\mathcal{A}$.
2. *Query Phase 1*: $\mathcal{A}$ can query:
 - $\mathcal{O}_{H_1}(w)$
 - $\mathcal{O}_{\text{skac}}(\text{ID}, S_{\text{ID}})$
 - $\mathcal{O}_{\text{Trapdoor}}(\text{sk}_{\text{du}}, W')$
3. *Challenge*: $\mathcal{A}$ submits w_0, w_1 and $\{v_{0,i}\}, \{v_{1,i}\}$. $\mathcal{C}$ picks $b \in \{0, 1\}$, generates I^* for $(w_b, \{v_{b,i}\})$, and sends I^* to $\mathcal{A}$.
4. *Query Phase 2*: $\mathcal{A}$ continues queries, but cannot query:
 - $\mathcal{O}_{H_1}(w_0)$ or $\mathcal{O}_{H_1}(w_1)$
 - $\mathcal{O}_{\text{Trapdoor}}$ for W' containing w_0 or w_1
 - $\mathcal{O}_{\text{skac}}$ for S_{ID} satisfying the challenge policy

5. *Guess*: $\mathcal{A}$ outputs $b' \in \{0, 1\}$.

The advantage is:

$$\text{Adv}_{\mathcal{A}}^{\text{IND-KGA-AVGA}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

Π is secure if this advantage is negligible.

Definition 3 (Trapdoor IND-KGA-AVGA Security) Let Π be an R-ABKS scheme and $\mathcal{A}$ be a PPT adversary. The trapdoor IND-KGA-AVGA security is defined via the following game:

1. *Initialization*: $\mathcal{A}$ prepares for the challenge. $\mathcal{C}$ runs $\text{Setup}(\lambda)$ to obtain (MPK, MSK) and provides MPK to $\mathcal{A}$.
2. *Query phase*: $\mathcal{A}$ may adaptively query:
 - $\mathcal{O}_{H_1}(w)$: Hash oracle for keywords.
 - $\mathcal{O}_{\text{skac}}(\text{ID}, S_{\text{ID}})$: User secret key generation.
 - $\mathcal{O}_{\text{IndexGen}}(w_j)$: Index generation for keywords.
 - $\mathcal{O}_{\text{Trapdoor}}(\text{sk}_{\text{du}}, W')$: Trapdoor generation for keyword set W' .
3. *Challenge*: $\mathcal{A}$ submits two challenge keyword sets $W'_0 = \{w_{0,j}\}_{j=1}^{m'_0}$ and $W'_1 = \{w_{1,j}\}_{j=1}^{m'_1}$, and two distinct attribute value sets $\{v_{0,i}\}_{i=1}^h$ and $\{v_{1,i}\}_{i=1}^h$. $\mathcal{C}$ randomly chooses $b \in \{0, 1\}$, generates the trapdoor $\text{Td}_{W'_b}$ for $(W'_b, \{v_{b,i}\}_{i=1}^h)$, and sends it to $\mathcal{A}$.
4. *Guess*: $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$.

The adversary $\mathcal{A}$ wins the game if $b' = b$ and the submitted challenge sets satisfy:

- $W'_0, W'_1 \notin L_W$ (not previously queried for trapdoors)
- $\{v_{0,i}\}_{i=1}^h, \{v_{1,i}\}_{i=1}^h \notin AS$ (not in any previously queried attribute set)

The advantage is defined as:

$$\text{Adv}_{\mathcal{A}}^{\text{Trapdoor-IND-KGA-AVGA}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

Π provides trapdoor IND-KGA-AVGA security if this advantage is negligible for all PPT adversaries $\mathcal{A}$.

3.5 Security assumptions

Definition 4 (q-Parallel BDHE Assumption) Let $\mathbb{G}$ and $\mathbb{G}_T$ be cyclic groups of prime order p with generator g , and $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ a bilinear map. For random $a, s \xleftarrow{\$} \mathbb{Z}_p^*$, given

$$\vec{y} = (g, g^s, g^a, \dots, g^{a^q}, g^{a^{q+2}}, \dots, g^{a^{2q}}) \in \mathbb{G}^{2q+1},$$

no PPT algorithm can distinguish $T = e(g, g)^{a^{q+1}s}$ from random $R \in \mathbb{G}_T$.

The advantage of $\mathcal{B}$ is ε if

$$|\Pr[\mathcal{B}(\vec{y}, T) = 0] - \Pr[\mathcal{B}(\vec{y}, R) = 0]| \geq \varepsilon.$$

The assumption holds if ε is negligible for all PPT adversaries.

Definition 5 (CDH Assumption) Let $\mathbb{G}$ be a group of prime order p with generator g . For $a, b \xleftarrow{\$} \mathbb{Z}_p^*$, given (g, g^a, g^b) , it is hard to compute g^{ab} .

$\mathcal{B}$ has advantage ε if

$$\Pr[\mathcal{B}(g, g^a, g^b) = g^{ab}] \geq \varepsilon.$$

The assumption holds if no PPT $\mathcal{B}$ achieves non-negligible ε .

Algorithm 1: System initialization

Input: Security parameter λ

Output: PP, MPK, MSK

- 1 Define G, G_T, p, g, e ;
 - 2 Select $H_0 : \{0, 1\}^* \rightarrow G, H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$;
 - 3 $PP = \{G, G_T, e, p, g, H_0, H_1\}$;
 - 4 **foreach** $v \in U$ **do**
 - 5 $\eta_v \leftarrow_R \mathbb{Z}_p, \text{PAK}_v = g^{\eta_v}$;
 - 6 $a, \alpha, \beta \leftarrow_R \mathbb{Z}_p^*$;
 - 7 $h(x) = \prod_{i=1}^n (x - H_1(\text{ID}_i))$
 - 8 $\text{MSK} = \{\alpha, a, \beta, \{\eta_v\}_{v \in U}\}$;
 - 9 $\text{MPK} = (g^a, g^\alpha, g^\beta, e(g, g)^a, \{\text{PAK}_v\}_{v \in U}, h(x))$;
 - 10 **return** $PP, \text{MPK}, \text{MSK}$
-

Algorithm 2: Key generation

1 User KeyGen:

Input: MSK, S_{ID}

Output: sk_{du}

- 2 $t \leftarrow_R \mathbb{Z}_p^*$;
- 3 $K_1 = g^t, K_2 = g^{at/\beta}, K_3 = g^{a+t}$;
- 4 **foreach** $v_i \in S_{\text{ID}}$ **do**
- 5 $K_i = H_0(v_i)^{t/\eta_v}$;
- 6 **return** $(K_1, K_2, K_3, \{K_i\})$

7 DO KeyGen:

Input: MSK

Output: $(\text{pk}_{\text{do}}, \text{sk}_{\text{do}})$

- 8 $r \leftarrow_R \mathbb{Z}_p^*$;
 - 9 **return** (g^r, r)
-

Algorithm 3: Data encryption

Input: MPK, $k_\theta, F = \{F_\theta\}, \mathbb{A} = (M, \rho)$

Output: CT, I_W

- 1 $C_\theta = \text{Enc}_{k_\theta}(F_\theta)$;
 - 2 $\vec{v} = (s, y_2, \dots, y_n)^T, y_i \leftarrow_R \mathbb{Z}_p^*$;
 - 3 **for** $i = 1$ **to** l **do**
 - 4 $\xi_i = M_i \cdot \vec{v}, \gamma_i \leftarrow_R \mathbb{Z}_p^*$;
 - 5 $CT_i = g^{a\xi_i} H_0(v_i)^{\gamma_i}$;
 - 6 $\hat{CT}_i = (g^{\eta_{\rho(i)}})^{\gamma_i}$;
 - 7 $CT_0 = k_\theta \cdot e(g, g)^{as}, \hat{CT}_0 = g^s$;
 - 8 $CT = (\mathbb{A}, C_\theta, CT_0, \hat{CT}_0, CT_i, \hat{CT}_i)$;
 - 9 $W = \{w_1, \dots, w_m\}$;
 - 10 **for** $i = 1$ **to** l **do**
 - 11 $\varepsilon_i \leftarrow_R \mathbb{Z}_p^*$;
 - 12 $I_i = g^{\eta_{v_i} \varepsilon_i}$;
 - 13 **for** $j = 1$ **to** m **do**
 - 14 $I_{i,j} = g^{a\xi_i} H_0(v_i)^{-\varepsilon_i/H_1(w_j)}$;
 - 15 $I_1 = g^{\beta s}$;
 - 16 $I_W = (I_1, \{I_i\}, \{I_{i,j}\})$;
 - 17 **return** CT, I_W
-

4 The specific construction of R-ABKS

This algorithm 1 is executed by a trusted third authority (TTA). The TA first generates common parameters based on the security parameter λ . Then, it generates random private key components and corresponding public key components for each attribute. In the core steps, the TA selects a random exponent to form the master private key and constructs the polynomial $h(x)$, setting the hashed identity of the registered user as its root. This lays the foundation for access control. Finally, the output includes the public parameters, the master public key (which is made public to all participants), and the master private key (which is kept secret by the TA and used for generating subsequent user private keys).

This algorithm 2 is executed by a trusted third authority (TTA), and its core function is to generate keys for data users (DU) and data owners (DO). It is a key part of the system's permission management. For DU, the TA takes the master private key and the DU attribute set as inputs, first selects a random number, then generates the core component bound to the master private key and the private key components corresponding to each attribute, and finally generates the DU private key that enables it to decrypt data or generate search tokens based on the attribute set. For DO, the TA only generates a simplified key pair containing the public key and private key for data encryption operations, ensuring that only users with legal attribute sets have the decryption capability.

This algorithm 3 is executed by DO and is used for data encryption and generation of searchable indexes. The DO first encrypts the data file using a symmetric key, then defines the access control policy described by the attribute matrix

Algorithm 4: Data search and decryption

1 TrapdoorGen:
Input: MPK, sk_{du} , W'
Output: $Td_{W'}$

2 $\varphi \leftarrow_R \mathbb{Z}_p^*$;
3 $T_1 = K_1^\varphi$, $T_2 = K_2^\varphi$;
4 **foreach** $w'_j \in W'$ and $v_i \in S_{ID}$ **do**
5 $T_{i,j} = K_i^{\varphi/H_1(w'_j)}$;
6 **return** $(T_1, T_2, \{T_{i,j}\})$

7 DataSearch:
Input: I_W , $Td_{W'}$, $\mathbb{A}$
Output: add

8 $I = \{i \mid \rho(i) \in S_{ID}\}$;
9 Solve $\sum_{i \in I} \omega_i M_i = (1, 0, \dots, 0)$ for $\{\omega_i\}$;
10 **if** $e(I_1, T_2) = \prod_{i \in I} \prod_{j=1}^{m'} (e(I_{i,j}, T_1) \cdot e(I_i, T_{i,j}))^{\omega_i}$ **then**
11 **return** add
12 **else**
13 **return** $\emptyset$

14 DataAccess:
Input: ID_i , add
Output: $CT_{W'}$ or reject

15 **if** $h(H_1(ID_i)) = 0$ **then**
16 **return** $CT_{W'}$, add
17 **else**
18 **return** reject

19 Decryption:
Input: $CT_{W'}$, C_θ , sk_{du}
Output: k_θ

20 $I = \{i \mid \rho(i) \in S_{ID}\}$;
21 $\sum_{i \in I} \omega_i \xi_i = s$;
22 **if** $\frac{\prod_{i \in I} e(CT_i, K_1)^{\omega_i}}{\prod_{i \in I} e(\hat{CT}_i, K_i)^{\omega_i}} = e(g, g)^{ats}$ **then**
23 **return** $k_\theta = \frac{CT_0 \cdot e(g, g)^{ats}}{e(K_3, CT_0)}$
24 **else**
25 **return** reject

and mapping function. Based on this, the symmetric key is encapsulated in the ciphertext header and attribute-related ciphertext components are generated.

This algorithm 4 requires interaction between DU and CSP, involving four sub-processes: The DU generates a trapdoor using the private key and the search keyword through TrapdoorGen and submits it to the CSP. The trapdoor can prove the search permission without revealing the key information; The CSP verifies the attributes of the DU and the keyword using bilinear pairing with the data search; If it passes, it returns the data address; The CSP then verifies the DU's identity through the initialization polynomial $h(x)$ to complete DataAccess; Finally, if the DU's attributes comply with the policy, it can decrypt the data by using the private key through Decryption.

Algorithm 5: Attribute revocation

Input: v' , $RL_{v'}$
Output: sk'_{du} , CT' , I'_W

1 $\hat{\eta}_{v'} \leftarrow_R \mathbb{Z}_p^*$;
2 $AUK_{v'} = \hat{\eta}_{v'} / \eta_{v'}$;
3 $\widehat{PAK}_{v'} = (PAK_{v'})^{AUK_{v'}}$;
4 **foreach** user $ID \notin RL_{v'}$ **do**
5 **foreach** $v_i \in S_{ID}$ **do**
6 **if** $v_i = v'$ **then**
7 $K'_i = K_i^{AUK_{v'}^{-1}}$
8 **else**
9 $K'_i = K_i$
10 $sk'_{du} = (K_1, K_2, K_3, \{K'_i\}')$
11 **foreach** $i = 1$ to l **do**
12 **if** $\rho(i) = v'$ **then**
13 $\hat{CT}'_i = (\hat{CT}_i)^{AUK_{v'}}$
14 **else**
15 $\hat{CT}'_i = \hat{CT}_i$
16 $CT' = (\mathbb{A}, C_\theta, CT_0, \hat{CT}_0, CT_i, \hat{CT}'_i)$;
17 **foreach** $i = 1$ to l **do**
18 **if** $\rho(i) = v'$ **then**
19 $I'_i = I_i^{AUK_{v'}}$
20 **else**
21 $I'_i = I_i$
22 $I'_W = (I_1, \{I_{i,j}\}, \{I'_i\})$;
23 **return** sk'_{du} , CT' , I'_W

This algorithm 5 is executed by a trusted third authority (TTA), enabling the dynamic revocation of the usage rights of specific user attributes. The TA first generates an attribute update key for the attribute to be revoked, then updates the relevant components of the attribute in all legal unrevoked user private keys; at the same time, it generates ciphertext update information to update the ciphertext and index components associated with the attribute stored in the cloud server (CSP), and the CSP completes the update of the stored content based on this. This process ensures that users whose attributes have been revoked cannot access subsequent encrypted data, and minimizes the impact on the data owner (DO) and other legitimate users to the greatest extent.

Correctness: (1) Search Correctness Proof

Left side of equation:

$$e(I_1, T_2) = e(g^{\beta s}, g^{at\varphi/\beta}) = e(g, g)^{ats\varphi}$$

Right side of equation:

$$\begin{aligned} & \prod_{i \in I} \prod_{j=1}^{m'} (e(I_{i,j}, T_1) \cdot e(I_i, T_{i,j}))^{\omega_i} \\ &= \prod_{i \in I} \prod_{j=1}^{m'} \left[e \left(g^{a\xi_i} H_0(v_i)^{-\varepsilon_i/H_1(w_j)}, g^{t\varphi} \right) \right] \end{aligned}$$

$$\begin{aligned}
& \cdot e \left(g^{\eta_v \varepsilon_i}, H_0(v_i)^{t\varphi/(\eta_v H_1(w_j))} \right) \Big] \Big]^{w_i} \\
&= \prod_{i \in I} \prod_{j=1}^{m'} \left[e(g, g)^{a \xi_i t \varphi} \cdot e(H_0(v_i), g)^{-\varepsilon_i t \varphi / H_1(w_j)} \right. \\
&\quad \left. \cdot e(g, H_0(v_i))^{\varepsilon_i t \varphi / H_1(w_j)} \right]^{w_i} \\
&= \prod_{i \in I} \prod_{j=1}^{m'} [e(g, g)^{a \xi_i t \varphi}]^{w_i} \\
&= e(g, g)^{at \varphi \sum_{i \in I} w_i \xi_i} = e(g, g)^{at s \varphi}
\end{aligned}$$

(2) Decryption Correctness Proof

$$\begin{aligned}
& \frac{CT_0 \cdot \prod_{i \in I} e(CT_i, K_1)^{w_i}}{e(K_3, \hat{CT}_0) \cdot \prod_{i \in I} e(\hat{CT}_i, K_i)^{w_i}} \\
&= \frac{k_\theta \cdot e(g, g)^{\alpha s} \cdot \prod_{i \in I} [e(g^{a \xi_i} H_0(v_i)^{\gamma_i}, g^t)]^{w_i}}{e(g^{\alpha+at}, g^s) \cdot \prod_{i \in I} [e(g^{\eta_v \gamma_i}, H_0(v_i)^{t/\eta_v})]^{w_i}} \\
&= \frac{k_\theta \cdot e(g, g)^{\alpha s} \cdot e(g, g)^{at \sum_{i \in I} w_i \xi_i} \cdot \prod_{i \in I} e(H_0(v_i), g)^{\gamma_i t w_i}}{e(g, g)^{\alpha s + at s} \cdot \prod_{i \in I} e(g, H_0(v_i))^{\gamma_i t w_i}} \\
&= \frac{k_\theta \cdot e(g, g)^{\alpha s + at s}}{e(g, g)^{\alpha s + at s}} = k_\theta
\end{aligned}$$

5 Security analysis

Theorem 1 Assuming the validity of the q -parallel Bilinear Diffie-Hellman Exponent (q -parallel BDHE) assumption, no probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ can succeed in Game-I with non-negligible probability. This demonstrates that the proposed revocable attribute-based keyword search (R-ABKS) scheme achieves indistinguishability under selective ciphertext-policy chosen-plaintext attack (IND-sCP-CPA) security.

Proof $\mathcal{A}^* = (\mathcal{M}^*, \rho^*)$, where $\mathcal{M}^*$ is an $l^* \times n^*$ matrix with $l^*, n^* \leq q$. Define: $\mathcal{U}^*$: Set of all attribute values in $\mathcal{A}^*$; For $v \in \mathcal{U}^*$, $I_v = \{i \in [1, l^*] \mid \rho^*(i) = v\}$ (row indices in $\mathcal{M}^*$ for attribute v); Constants $\{c_{i,j}\}_{i \in [1, l^*], j \in [1, n^*]}$ for constructing well-formed polynomials of $v \in \mathcal{U}^*$.

(1) Initialization. The challenger $\mathcal{C}$ receives a q -parallel BDHE instance:

$$\{g, g^s, g^a, g^{a^2}, \dots, g^{a^q}, g^{a^{q+2}}, \dots, g^{a^{2q}}, T\} \in \mathbb{G}^{2q+1} \times \mathbb{G}_T$$

where T is either $e(g, g)^{a^{q+1}s}$ or random in $\mathbb{G}_T$. The adversary $\mathcal{A}$ submits the challenge access structure $\mathcal{A}^* = (\mathcal{M}^*, \rho^*)$ to $\mathcal{C}$.

(2) Setup. $\mathcal{C}$ simulates system setup as follows:

1. **Master key parameter setup:** Randomly select $\alpha', \beta \in \mathbb{Z}_p$; implicitly set $\alpha = \alpha' + a^{q+1}$. Compute:

$$e(g, g)^\alpha = e(g, g)^{\alpha'} \cdot e(g^a, g^{a^q})$$

Note: $\mathcal{C}$ cannot compute g^α directly.

2. **Attribute public key generation:** For each $v \in U$, select $\eta_v \in \mathbb{Z}_p$ and compute $PAK_v = g^{\eta_v}$.

3. **Hash function H_0 Construction:** For each $v \in U$, select $\tilde{v} \in \mathbb{Z}_p$: - If $v \notin \mathcal{U}^*$: $H_0(v) = g^{\tilde{v}}$; - If $v \in \mathcal{U}^*$: Construct polynomial $P_v(x) = \tilde{v} + \sum_{i \in I_v} \sum_{j=1}^{n^*} c_{i,j} x^j$, then

$$H_0(v) = g^{P_v(a)} = g^{\tilde{v}} \cdot \prod_{i \in I_v} \prod_{j=1}^{n^*} (g^{a^j})^{c_{i,j}}$$

4. **Hash function H_1 Construction:** For each keyword w queried, select $h_w \in \mathbb{Z}_p$ randomly and set $H_1(w) = h_w$.

5. **Polynomial $h(x)$ Setup:** Select $H_1(\text{ID}_i) \in \mathbb{Z}_p$ for n distinct user identities ID_i , then $h(x) = \prod_{i=1}^n (x - H_1(\text{ID}_i))$.

6. **Public parameters publication:**

$$\text{MPK} = (PP, g^a, g^\beta, e(g, g)^\alpha, \{PAK_v\}_{v \in U}, h(x))$$

$\mathcal{C}$ implicitly maintains $\text{MSK} = \{\alpha, a, \beta, \{\eta_v\}_{v \in U}\}$ without knowing a or full α .

(3) Query Phase 1. $\mathcal{C}$ maintains a key list $\mathcal{L}_{sk}$. $\mathcal{A}$ makes queries with S_{ID} not satisfying $\mathcal{A}^*$:

1. **User key query $\text{sk}_{\text{du}}(\text{ID}, S_{ID})$:** By LSSS property, there exists $\vec{w} = (w_1, \dots, w_{n^*}) \in \mathbb{Z}_p^{n^*}$ with $w_1 = -1$ and $\mathcal{M}_i^* \cdot \vec{w} = 0$ for all i with $\rho^*(i) \in S_{ID}$. $\mathcal{C}$ sets $t = r - \sum_{j=1}^{n^*} w_j a^{q+1-j}$ ($r \in \mathbb{Z}_p$ random) and computes:

$$\begin{aligned}
K_1 &= g^t = g^r \cdot \prod_{j=1}^{n^*} (g^{a^{q+1-j}})^{-w_j} \\
K_2 &= g^{at/\beta} = (g^a)^{r/\beta} \cdot \prod_{j=1}^{n^*} (g^{a^{q+2-j}})^{-w_j/\beta} \\
K_3 &= g^{\alpha+at} = g^{\alpha'+a^{q+1}+a(r-\sum_{j=1}^{n^*} w_j a^{q+1-j})} \\
&= g^{\alpha'} \cdot g^{ar} \cdot \prod_{j=1}^{n^*} (g^{a^{q+2-j}})^{-w_j}
\end{aligned}$$

For attribute key components ($v_i \in S_{ID}$): If $v_i \notin \mathcal{U}^*$: $K_i = H_0(v_i)^{t/\eta_v} = (K_1)^{\tilde{v}_i/\eta_v}$; If $v_i \in \mathcal{U}^*$: $K_i = H_0(v_i)^{t/\eta_v} = (g^{P_{v_i}(a)})^{t/\eta_v}$.

2. **Attribute revocation key query $\text{AUK}(v')$:** Select $\hat{\eta}_{v'} \in \mathbb{Z}_p$, return $\text{AUK}_{v'} = \hat{\eta}_{v'}/\eta_{v'}$.

3. **Data owner key query sk_{do} :** Select $r \in \mathbb{Z}_p$, return $(pk_{\text{do}}, sk_{\text{do}}) = (g^r, r)$.

4. *Trapdoor query* $\mathcal{O}_{\text{Trapdoor}}(\text{sk}_{\text{du}}, W')$: For keyword set W' not containing challenge keywords, select $\varphi \in \mathbb{Z}_p$ and compute:

$$T_1 = K_1^\varphi, \quad T_2 = K_2^\varphi, \quad T_{i,j} = K_i^{\varphi/H_1(w'_j)}$$

(4) **Challenge.** $\mathcal{A}$ submits two equal-length messages k_0, k_1 . $\mathcal{C}$ proceeds:

1. Select $\sigma \in \{0, 1\}$; use the BDHE instance's g^s as the ciphertext randomness.

2. Construct core ciphertext components:

$$CT_0^* = k_\sigma \cdot T \cdot e(g, g)^{\alpha's}, \quad \widehat{CT}_0^* = g^s$$

3. Construct attribute-related components: Select $y_2, \dots, y_{n^*} \in \mathbb{Z}_p$, set $\vec{v} = (s, sa + y_2, sa^2 + y_3, \dots, sa^{n^*-1} + y_{n^*})^T$. For $i = 1$ to l^* :

$$\begin{aligned} \xi_i &= \mathcal{M}_i^* \cdot \vec{v} = \sum_{j=1}^{n^*} \mathcal{M}_{i,j}^* (sa^{j-1} + y_j) \\ CT_i^* &= g^{a\xi_i} \cdot H_0(\rho^*(i))^{\gamma_i} \\ &= \prod_{j=1}^{n^*} (g^{a^j})^{s \cdot \mathcal{M}_{i,j}^*} \cdot \prod_{j=1}^{n^*} (g^{a^j})^{y_j \cdot \mathcal{M}_{i,j}^*} \cdot H_0(\rho^*(i))^{\gamma_i} \\ \widehat{CT}_i^* &= g^{\eta_{\rho^*(i)} \gamma_i} \end{aligned}$$

4. Construct search index I_w^* :

$$\begin{aligned} I_1^* &= g^{\beta s} \\ I_i^* &= g^{\eta_{\rho^*(i)} \varepsilon_i} \quad \text{for } i = 1 \text{ to } l^* \\ I_{i,j}^* &= g^{a\xi_i} \cdot H_0(\rho^*(i))^{-\varepsilon_i/H_1(w_j)} \quad \text{for each keyword } w_j \end{aligned}$$

5. Output challenge ciphertext $CT^* = (\mathcal{A}^*, CT_0^*, \widehat{CT}_0^*, \{CT_i^*, \widehat{CT}_i^*\}_{i=1}^{l^*})$ and I_w^* .

(5) **Query Phase 2.** $\mathcal{A}$ continues queries with the same restrictions as Phase 1.

(6) **Guess.** $\mathcal{A}$ submits σ' (guess for σ): If $\sigma = \sigma'$, $\mathcal{C}$ outputs 0 (concludes $T = e(g, g)^{a^{q+1}s}$); Else, $\mathcal{C}$ outputs 1 (concludes T is random in $\mathbb{G}_T$).

Advantage analysis. We analyze $\mathcal{C}$'s advantage in solving q-parallel BDHE:

Case 1: $T = e(g, g)^{a^{q+1}s}$: The core component becomes: $CT_0^* = k_\sigma \cdot e(g, g)^{a^{q+1}s} \cdot e(g, g)^{\alpha's} = k_\sigma \cdot e(g, g)^{(\alpha' + a^{q+1})s} = k_\sigma \cdot e(g, g)^{\alpha's}$. This is consistent with the real scheme. $\mathcal{A}$ distinguishes with advantage ε , so $\Pr[\mathcal{C} \rightarrow 0 \mid T = e(g, g)^{a^{q+1}s}] = \frac{1}{2} + \varepsilon$.

Case 2: T random in $\mathbb{G}_T$: CT_0^* is statistically independent of k_σ , so $\mathcal{A}$ gains no advantage: $\Pr[\mathcal{C} \rightarrow 0 \mid T \text{ random}] = \frac{1}{2}$.

Overall advantage: $\mathcal{C}$'s advantage is $\mathcal{A} = \left| \left(\frac{1}{2} + \varepsilon \right) - \frac{1}{2} \right| = \varepsilon$.

If ε is non-negligible, this contradicts the q-parallel BDHE assumption. Thus, the R-ABKS scheme is IND-sCP-CPA secure. $\square$

Theorem 2 *If the computational Diffie-Hellman (CDH) problem is hard, then no probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ can succeed in Game-II with more than a negligible advantage. Consequently, the keyword index construction presented herein achieves security under the indistinguishability against keyword guessing and attribute-value guessing attacks (IND-KGA-AVGA).*

Proof Suppose a PPT adversary $\mathcal{A}$ can, with non-negligible probability, distinguish between keywords and their corresponding attribute values within the index. We then build a simulator $\mathcal{C}$ that leverages $\mathcal{A}$'s capability to solve an instance of the CDH problem.

(1) **Initialization.** The simulator $\mathcal{C}$ performs the system setup as follows:

1. Choose random elements a, α, β from $\mathbb{Z}_p^*$. 2. Model the hash functions as random oracles: $H_0 : \{0, 1\}^* \rightarrow \mathbb{G}$ and $H_1 : \{0, 1\}^* \rightarrow \mathbb{Z}_p^*$. 3. For every possible attribute value v in the universe U , pick a random $\eta_v \in \mathbb{Z}_p$ and set the public attribute key as $PAK_v = g^{\eta_v} = g^d$. 4. Generate a polynomial $h(x) = \prod_{i=1}^n (x - H_1(\text{ID}_i))$ over a set of n user identifiers. 5. The public parameters are published as:

$$\text{MPK} = \{g^a, g^\alpha, g^\beta, e(g, g)^a, \{PAK_v \mid v \in U\}, h(x)\},$$

while the master secret key $\text{MSK} = (\alpha, a, \beta, \{\eta_v\}_{v \in U})$ is retained privately.

(2) **Query phase 1.** The adversary $\mathcal{A}$ adaptively issues a series of queries, which $\mathcal{C}$ answers as described below. $\mathcal{A}$ provides a collection of attribute sets and a keyword set $W^* = \{w_1, \dots, w_m\}$.

1. *Hash query* $\mathcal{O}_{H_0}(v)$: For an attribute value v , $\mathcal{C}$ checks a maintained list L_v . If v is not found, it selects a random $\tilde{v} \in \mathbb{Z}_p$, stores $(v, \tilde{v})$ in L_v , and returns $H_0(v) = g^{\tilde{v}}$. Otherwise, it returns the previously stored value.

2. *Hash query* $\mathcal{O}_{H_1}(w)$: For a keyword w , $\mathcal{C}$ consults a list L_w . If w is new, it picks a random $\tilde{w} \in \mathbb{Z}_p$, records $(w, \tilde{w})$ in L_w , and sets $H_1(w) = \tilde{w}$. The stored value is returned for repeated queries.

3. *User secret key query* $\mathcal{O}_{\text{skac}}(\text{ID}, S_{ID})$: Given a user identifier ID and an attribute set S_{ID} , $\mathcal{C}$ selects a random $t \in \mathbb{Z}_p^*$ and computes the secret key components:

$$\begin{aligned} K_1 &= g^t, \\ K_2 &= g^{at/\beta}, \\ K_3 &= g^{\alpha+at}, \\ K_i &= H_0(v_i)^{t/\eta_{v_i}} = g^{\tilde{v}_i t / \eta_{v_i}} \quad \text{for each attribute } v_i \in S_{ID}. \end{aligned}$$

The tuple $sk_{du} = (K_1, K_2, K_3, \{K_i\}_{v_i \in S_{ID}})$ is stored in a list L_{sk} and given to $\mathcal{A}$.

4. *Index generation query*: For the keyword set $w_j \subseteq W^*$, $\mathcal{C}$ chooses random $s, y_2, \dots, y_n \in \mathbb{Z}_p^*$ and defines the vector $\vec{v} = (s, y_2, \dots, y_n)^T$. For each row i (from 1 to l) of the access matrix M , it computes $\xi_i = M_i \cdot \vec{v}$, picks a random $\varepsilon_i \in \mathbb{Z}_p^*$, and constructs:

$$\begin{aligned} I_1 &= g^{\beta s}, \\ I_i &= g^{\eta v \varepsilon_i}, \\ I_{i,j} &= g^{a \xi_i} \cdot H_0(v_i)^{-\varepsilon_i / H_1(w_j)} = g^{a \xi_i} g^{-\tilde{v}_i \varepsilon_i / \tilde{w}_j}. \end{aligned}$$

The resulting index $I_w = (I_1, \{I_i\}_{i=1}^l, \{I_{i,j}\}_{i,j})$ is returned.

5. *Trapdoor query*: For a subset of keywords $w_j \subseteq W^*$, $\mathcal{C}$ selects a random $\varphi \in \mathbb{Z}_p^*$ and generates the trapdoor:

$$\begin{aligned} T_1 &= K_1^\varphi = g^{t\varphi}, \\ T_2 &= K_2^\varphi = g^{at\varphi/\beta}, \\ T_{i,j} &= K_i^{\varphi/H_1(w_j)} = g^{\tilde{v}_i t \varphi / (\eta_{v_i} \tilde{w}_j)}. \end{aligned}$$

The trapdoor $Td_{W'} = (T_1, T_2, \{T_{i,j}\})$ is sent to $\mathcal{A}$.

(3) **Challenge**. $\mathcal{A}$ submits two distinct keywords w_0, w_1 of equal length, along with two sets of attribute values $\{v_{0,i}\}_{i=1}^{l^*}$ and $\{v_{1,i}\}_{i=1}^{l^*}$, each associated with a different access policy. $\mathcal{C}$ randomly chooses a bit $b \in \{0, 1\}$ and prepares the challenge as follows:

1. For $i = 1$ to l^* , select random $\varepsilon_i \in \mathbb{Z}_p^*$.
2. Compute the core index components:

$$\begin{aligned} I_1 &= g^{\beta s}, \\ I_i &= g^{\eta v \varepsilon_i}, \\ I_{i,j}^b &= g^{a \xi_i} \cdot H_0(v_{b,i})^{-\varepsilon_i / H_1(w_{b,j})} = g^{a \xi_i} g^{-\tilde{v}_{b,i} \varepsilon_i / \tilde{w}_{b,j}}. \end{aligned}$$

3. Choose random $v_0, v_1 \in \mathbb{Z}_p^*$ and compute auxiliary challenge values:

$$\begin{aligned} A_0 &= (g^d)^{v_0}, \quad A_1 = (g^d)^{v_1}, \\ B_0 &= \left\{ \left(H_0(v_{0,i})^{1/H_1(w_{0,j})} \right)^{v_0} \right\}_{i=1}^{l^*}, \\ B_1 &= \left\{ \left(H_0(v_{1,i})^{1/H_1(w_{1,j})} \right)^{v_1} \right\}_{i=1}^{l^*}. \end{aligned}$$

4. The challenge tuple $(I_1, \{I_i, I_{i,j}^b\}_{i=1}^{l^*}, v_b, A_b, B_b)$ is given to $\mathcal{A}$.

Constraints: The challenge keyword w_b must not have been queried to H_1 , and none of the attribute sets $\{v_{b,i}\}_{i=1}^{l^*}$ can have been used in a prior secret key query.

(4) **Query Phase 2**. $\mathcal{A}$ may continue to issue queries as in Phase 1, subject to the same constraints regarding the challenge data.

(5) **Guess**. Finally, $\mathcal{A}$ outputs an element $Z \in \mathbb{G}$ and a guess $b' \in \{0, 1\}$. $\mathcal{A}$ is considered to win if the following verification holds:

$$e(I_1, Z) = \prod_{i=1}^{l^*} \left(e(I_{i,j}^{b'}, A_{b'}) \cdot e(I_i, B_{b'}) \right)^{\omega_i},$$

where ω_i are appropriate reconstruction coefficients. If $\mathcal{A}$ wins, $\mathcal{C}$ extracts the solution to the CDH instance as:

$$g^{ad} = Z^{\beta/v_{b'}}.$$

Advantage analysis. Should $\mathcal{A}$ possess a non-negligible advantage ε in distinguishing the challenge keyword and attribute values, then $\mathcal{C}$ can leverage $\mathcal{A}$ to compute g^{ad} from the given triple (g, g^a, g^d) with the same advantage ε , thereby violating the CDH assumption. Given the hardness of the CDH problem, $\mathcal{A}$'s advantage must be negligible. Hence, the proposed scheme's keyword index provides IND-KGA-AVGA security. $\square$

Theorem 3 No PPT adversary $\mathcal{A}$ can win Game-III with non-negligible advantage. The trapdoor is indistinguishable under keyword guessing attacks (KGA) and attribute value guessing attacks (AVGA), which means the trapdoor in the proposed scheme is IND-KGA-AVGA secure.

Proof Assume there exists a PPT adversary $\mathcal{A}$ that can successfully distinguish keywords and attribute values in trapdoors. In Game-III, we construct a challenger $\mathcal{C}$ to interact with $\mathcal{A}$.

(1) **Initialization**. The procedure is consistent with Game-II.

(2) **Query Phase** The procedure is consistent with Game-II.

(3) **Challenge**. $\mathcal{A}$ submits two challenge keyword sets $W'_b = \{w_{b,j}\}_{j=1}^{m'}$ and two distinct attribute value sets $\{v_{b,i}\}_{i=1}^h$, where $b \in \{0, 1\}$, and sends them to $\mathcal{C}$. $\mathcal{C}$ flips a fair coin and constructs the trapdoor for W'_b and $\{v_{b,i}\}_{i=1}^h$. $\mathcal{C}$ randomly selects $\varphi \in \mathbb{Z}_p^*$ and computes:

$$\begin{aligned} T_1 &= K_1^\varphi = g^{t\varphi}, \quad T_2 = K_2^\varphi = g^{at\varphi/\beta}, \\ T_{i,j}^b &= (K_i)^{\varphi/H_1(w_{b,j})} = g^{\tilde{v}_{b,i} t \varphi / (\eta_{v_{b,i}} \tilde{w}_{b,j})}. \end{aligned}$$

$\mathcal{C}$ sends the trapdoor $Td_{W'_b} = (T_1, T_2, \{T_{i,j}^b\}_{i=1}^h)$ to $\mathcal{A}$, where $b \in \{0, 1\}$. The constraints are $W'_b \notin L_W$ and $\{v_{b,i}\}_{i=1}^h \notin AS$.

(4) **Guess**. If $\mathcal{A}$ finds $Z \in \mathbb{G}$ satisfying the discriminant

$$e(Z, T_{i,j}^{b'}) = e(T_1, B_{b'}),$$

where $b' \in \{0, 1\}$, then $\mathcal{A}$ wins Game-III. $\mathcal{A}$ sends Z and b' to $\mathcal{C}$. $\mathcal{C}$ computes $Z = g^{d \cdot \tilde{v}_{b'} / \eta_{v_{b'}}}$.

Since the system master key component η_v contained in the trapdoor $T_{i,j}^b$ is not accessible to $\mathcal{A}$, $\mathcal{A}$ cannot infer

attribute values or keyword information from the trapdoor based on whether the discriminant holds. $\square$

6 Performance analysis

6.1 Functional comparison

To clearly delineate the advancements of our proposed scheme, we present a systematic functional comparison with several state-of-the-art ciphertext-policy attribute-based keyword search schemes, namely EMKV-ABSE [18], MKSVABE [19], ABMKSE [20], ABEROD [22], and EMK-ABSE [23]. The comparison is conducted across four critical criteria: Multi-keyword Search, Multi-user Setting, Support for Linear Secret Sharing Scheme (LSSS), and Attribute Revocation. The results are summarized in Table 2.

As illustrated in Table 2, all the surveyed schemes, including our own, successfully support multi-keyword search, operate in a multi-user environment, and employ LSSS for constructing flexible and expressive access policies. This demonstrates that these features have become foundational and widely adopted in modern secure searchable encryption systems. The capability for LSSS, in particular, is crucial as it enables the implementation of sophisticated fine-grained access control with complex Boolean logic, which is a prerequisite for practical applications in cloud environments.

The most significant point of differentiation lies in the support for attribute revocation. Efficient and secure attribute revocation is paramount in real-world dynamic systems where user roles and permissions frequently change (e.g., employee departure or role change). As shown in the table, schemes like EMKV-ABSE, MKSVABE, ABEROD, and EMK-ABSE lack this critical functionality. The absence of a revocation mechanism in these schemes poses a serious security risk, as users would retain their access privileges indefinitely even after their attributes have been invalidated, leading to potential unauthorized data access.

In contrast, our proposed scheme and the ABMKSE scheme both incorporate a robust attribute revocation mech-

anism. This feature ensures that the system can promptly and efficiently revoke a user's access rights upon attribute changes, thereby maintaining the forward secrecy of the outsourced data and enhancing the overall security posture. Therefore, while our scheme shares the core functionalities with its peers, it distinguishes itself from the majority (EMKV-ABSE, MKSVABE, ABEROD, EMK-ABSE) by offering this essential security feature. This positions our scheme, alongside ABMKSE, as a more complete and practical solution for dynamic and security-sensitive cloud storage scenarios.

6.2 Theoretical performance

This subsection provides a theoretical analysis of the computational and storage overhead associated with our proposed scheme in comparison to existing ones. The analysis aims to highlight the intrinsic efficiency characteristics of each scheme before proceeding to empirical evaluation.

A comprehensive theoretical analysis of computational costs is conducted to evaluate the efficiency of the proposed scheme against existing approaches: EMKV-ABSE, MKSVABE, ABMKSE, and EMK-ABSE. As summarized in Table 3, the analysis employs standardized notation where E , P , and E_T represent the costs of exponentiation, pairing, and target group exponentiation operations, respectively. The encryption cost of ours is $(2l + lm)E + E_T$, demonstrating linear scalability with both policy attributes (l) and keywords (m), while being dominated by relatively efficient exponentiation operations. For trapdoor generation, ours requires $(2 + |S|m')E$, maintaining linear dependence on user attributes ($|S|$) and query keywords (m'), comparable to ABMKSE and MKSVABE. Notably, EMKV-ABSE achieves enhanced client efficiency by completely outsourcing trapdoor generation (**FE**) to edge nodes. The search phase reveals distinctive characteristics: ours incurs $(2lm')P$ cost, scaling with the product of policy attributes and query keywords but consisting solely of pairing operations suitable for cloud server execution. EMK-ABSE demonstrates exceptional efficiency with a constant search cost of $4P$, while MKSVABE exhibits complex dependency on multiple parameters including attribute value summation ($\sum U_r$). For decryption, ours shows $(2|I| + 1)P + E_T$ cost, linear in satisfying attributes ($|I|$), and shares with EMKV-ABSE the strategic advantage of outsourcing computationally intensive components to external entities, significantly reducing client-side burden. This analysis reveals that ours provides a balanced performance profile with predictable linear scaling characteristics and efficient outsourcing strategies, making it particularly suitable for practical deployments with resource-constrained user devices.

A comparative analysis of the storage overhead for critical system components is delineated in Table 4, where the

Table 2 Feature comparison

Scheme	Multi-keyword	Multi-user	LSSS	Attribute revocation
EMKV-ABSE [18]	✓	✓	✓	×
MKSVABE [19]	✓	✓	✓	×
ABMKSE [20]	✓	✓	✓	✓
ABEROD [22]	✓	✓	✓	×
EMK-ABSE [23]	✓	✓	✓	×
Our scheme	✓	✓	✓	✓

Table 3 Comparison of computational cost

Scheme	Encryption	Trapdoor	Search	Decryption
EMKV-ABSE	$3\zeta P$	FE	$(2\zeta + 1)P + \zeta E$	CP + $(1 + 2\zeta)E$
MKSVABE	$(3l + 3)E + E_T$	$(S j + 2)E$	$(\sum U_r + \dots)E + 3E_T$	$(2U + 1)P + E_T$
ABMKSE	$(3l + 3)E + E_T$	$(S j + 2)E$	$(2lj + 1)P + lE_T$	$(2l + 1)P + 2E_T$
EMK-ABSE	$(2 U + 1 + l)E + E_T + P$	$(j + 3)E$	$4P$	$E + (2l + 2)P + 3E_T$
Ours	$(2l + lm)E + E_T$	$(2 + S m')E$	$(2lm')P$	$(2 I + 1)P + E_T$

Costs marked in bold indicate operations that are outsourced to an external entity (e.g., a cloud server or edge node). Key parameters: ζ , l , U (number of attributes); $|S|$ (user's attribute set size); F , m , m' , j (number of keywords); $|I|$ (number of attributes satisfying the policy)

Table 4 Comparison of storage overhead

Scheme	Ciphertext	Private key	Index	Trapdoor
EMKV-ABSE	$(2 + \zeta) G + G_T $	$ Z_p $	$3 G $	$3 G $
MKSVABE	$O(\sum U_r + m + f) G $	$O(f + U) G $	–	$O(U) G $
EMK-ABSE	$(5 + 2 S + j) G $	$(4 + S) G + Z_p^* $	–	–
Ours	$O(l) G $	$O(S) G $	$O(l + l \cdot m) G $	$O(S \cdot m') G $

ζ , l , U , attributes in policy/system; $|S|$, user attributes; m , m' , keywords in file/query; f , data owners; $\sum U_r$, total attribute values; j , keyword-related components

notation $|G|$, $|G_T|$, and $|Z_p|$ denotes the bit-length of an element in the base group, target group, and finite field, respectively. The analysis reveals distinct efficiency characteristics among the schemes. EMKV-ABSE benefits from a constant-size index and trapdoor, a significant advantage for communication efficiency, though its ciphertext size grows linearly with the number of attributes ζ . In stark contrast, MKSVABE exhibits the most complex and potentially burdensome storage overhead. Its ciphertext and private key sizes depend on multiple system-wide parameters, including the total number of attribute values ($\sum U_r$) and data owners (f), which raises substantial scalability concerns in large, dynamic systems. EMK-ABSE employs a different architectural approach, integrating functionalities such that it lacks discrete index and trapdoor structures, but its ciphertext and private key sizes are explicitly defined and scale with user attributes and keyword components. Our proposed scheme demonstrates a predictable and highly scalable storage model. The ciphertext and private key sizes scale linearly solely with the number of attributes in the policy (l) and the user's attribute set ($|S|$), respectively. Furthermore, the index and trapdoor sizes grow linearly with the product of the number of attributes and keywords, which is an intuitive and manageable growth pattern. This linear scaling with directly controllable parameters is generally preferable to the complex, system-wide dependencies seen in MKSVABE, establishing our scheme as offering a balanced and efficient profile suitable for practical, resource-constrained environments.

6.3 Experimental analysis

We conducted some experiments on a platform equipped with an Apple M2 chip, utilizing the PBC library implemented in C++. The tests measure the computational overhead of the key seven stages. For a comprehensive comparison, the performance of our scheme was benchmarked against EMKV-ABSE, MKSVABE, and ABEROD under identical conditions. The system attribute universe size $|U|$ was varied from 100 to 400 for the setup phase, while the number of attributes in a user's key or an access policy (l or $|S|$) was varied from 4 to 10 for other phases. Each operation was executed multiple times, and the average running time (in milliseconds) is reported.

The setup time (Fig. 2) for all schemes grows linearly with the attribute universe size $|U|$. Our scheme demonstrates highly competitive performance, matching or slightly outperforming EMKV-ABSE and MKSVABE across the tested range. At $|U| = 400$, our scheme completes in 0.30 ms.

Figure 3 highlights a significant advantage of our scheme. Our key generation time is substantially lower than that of MKSVABE and is also more efficient than both EMKV-ABSE and ABEROD. For a user with 10 attributes, our scheme requires only 0.05 ms, which is over 8 times faster than MKSVABE(0.43 ms).

During the encryption stage (Fig. 4), our solution has moderate costs and exhibits linear growth, and its performance is superior to that of ABEROD, but slightly inferior to the highly optimized EMKV-ABSE solution. The reason is that the EMKV-ABSE solution uses offline encryption. However, in the index generation stage (Fig. 5), as the number

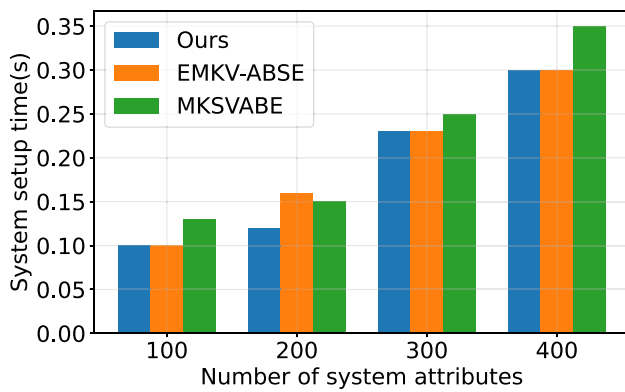


Fig. 2 Comparison of system setup performance

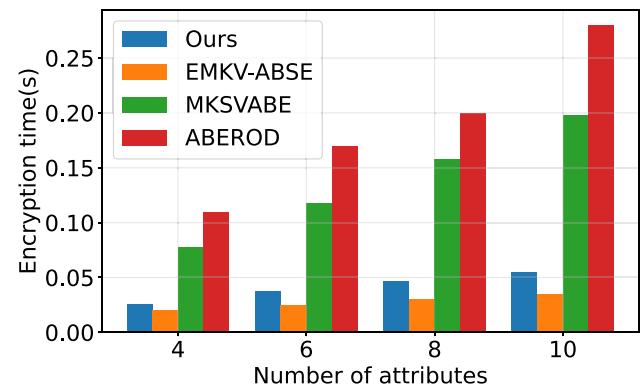


Fig. 4 Comparison of encryption performance

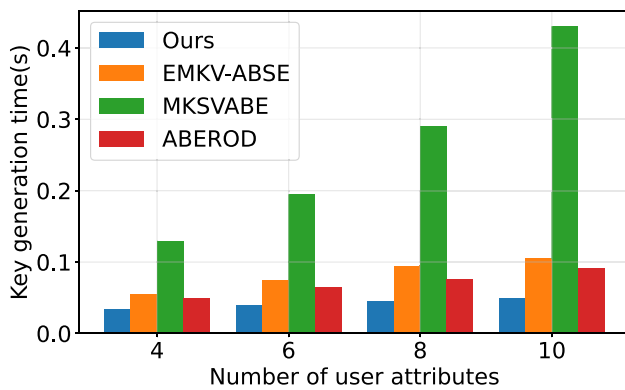


Fig. 3 Comparison of key generation performance

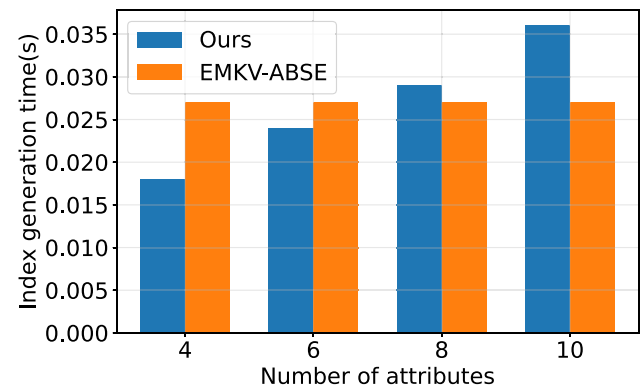


Fig. 5 Comparison of index generation performance

of attributes exceeds 4, our solution becomes more efficient than EMKV-ABSE.

(According to the data in Fig. 6) During this stage, the efficiency of generating the trapdoor in this scheme is particularly outstanding. Experimental data shows that the trapdoor generation time of this scheme remains relatively stable under different attribute quantities, and is lower than that of the EMKV-ABSE and MKSVABE schemes. Specifically, under the same testing conditions, the trapdoor generation time of this scheme is approximately one-tenth of that of the EMKV-ABSE scheme, demonstrating a certain efficiency advantage.

The search time (Fig. 7) for our scheme is constant and the lowest among all, at 0.006 ms, demonstrating that the search efficiency is independent of the number of attributes. This might help to reduce the computational load on the server side.

Figure 8 presents the cost of attribute revocation in our scheme, which scales reasonably with the number of attributes. This measured overhead is the practical cost for achieving dynamic access control, a feature not supported by all compared schemes.

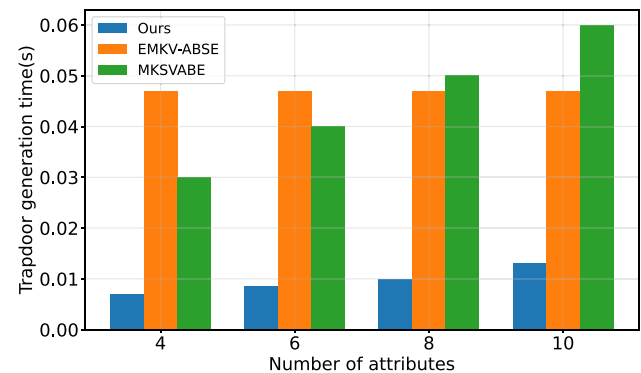


Fig. 6 Comparison of trapdoor generation performance

7 Conclusion and future work

This paper presents a novel searchable attribute-based encryption (ABE) scheme that effectively addresses the challenge of attribute revocation in cloud storage. The proposed solution uniquely integrates multi-keyword search capability with an efficient revocation mechanism. Its core innovation is enabling immediate attribute revocation without requiring data re-encryption, while maintaining robust security. Performance evaluations demonstrate significant efficiency

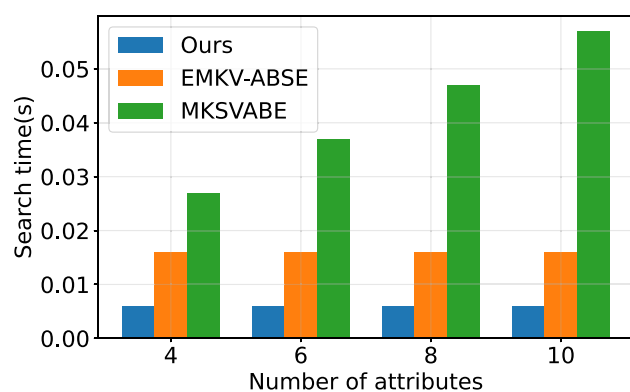


Fig. 7 Comparison of search performance

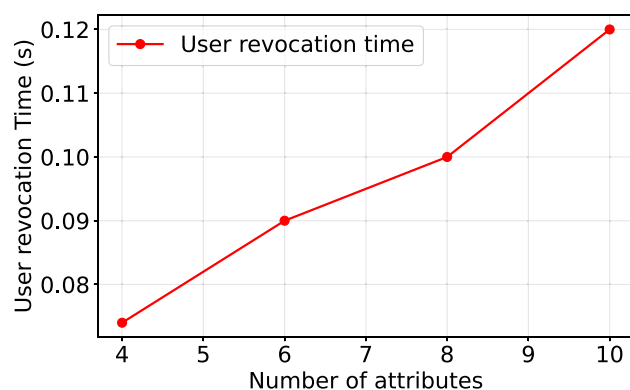


Fig. 8 Attribute revocation time overhead

improvements, particularly in key and trapdoor generation, making it practical for real-world deployment.

The current design has several constraints. First, the exposure of access policies poses a privacy risk, as sensitive attribute information is not hidden. Second, security depends entirely on a single trusted Attribute Authority, creating a central point of failure. Third, the revocation mechanism lacks forward secrecy, allowing revoked users potential access to previously encrypted data. Finally, the scheme does not verify the integrity of search results, leaving it vulnerable to malicious server behavior.

To overcome these limitations, future research will focus on: integrating policy-hiding techniques to protect attribute privacy; decentralizing trust through multi-authority architectures; enhancing revocation to achieve forward secrecy; and developing mechanisms for search result verification. We will also extend the scheme to support complex query types and optimize its scalability for large-scale distributed environments.

Acknowledgements The Haikou Science and Technology Special Fund (2420016000142), the Graduate Innovative Research Project of Hainan Province (Qhys2024-02), The Hainan Province Science and Technology Special Fund (ZDYF2025GXJS194).

Author Contributions Zixin Xiong.: Conceptualization, Methodology, Investigation, Data Curation, Software, Writing - Original Draft. Jun Ye: Conceptualization, Methodology, Validation, Formal Analysis, Writing - Original Draft, Writing - Review & Editing, Supervision, Funding Acquisition. All authors reviewed and approved the final manuscript.

Data Availability No datasets were generated or analysed during the current study.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Kesavan, E.: The impact of cloud computing on software development: a review[J]. *Int. J. Innov. Sci. Eng. Manag.* 269-274 (2025)
2. Vaigandla, K.K.: Data classification and emerging encryption technologies in big data and cloud computing: a systematic review[J]. *Appl. Data. Sci. Anal.* **2025**, 187–200 (2025)
3. Rezaeinejad, S., Rayman, K., Sauser, B., et al.: How can blockchain contribute to cyber resilience? Supply chain analytics and digitalization benefits[J]. *Inf. Syst. Front.* 1-12 (2025)
4. Wu, J., Xu, Y., Zhang, K., et al.: AuthGraph: Authorized search over encrypted social graph database with trusted hardware[J]. *IEEE Transactions on Dependable and Secure Computing*, (2025)
5. Wu, J., Xu, Y., Zhang, K., et al.: AuthGraph: authorized search over encrypted social graph database with trusted hardware[J]. *IEEE Transactions on Dependable and Secure Computing*, (2025)
6. Hassan, J., Qin, Z., Rauf, M.A., et al.: Revocable policy hiding bilateral access control scheme with equality test for IIoT environment[J]. *IEEE Internet of Things Journal*, (2025)
7. Yan, L., Wang, G., Yin, T., et al.: Attribute-based searchable encryption: a survey[J]. *Electronics* **13**(9), 1621 (2024)
8. Ali, M., Hongjie, H., Hussain, A., et al.: Towards verifiable updatable attribute-based Boolean keywords search sharing of PHRs in cloud environment[J]. *Clust. Comput.* **28**(13), 856 (2025)
9. Li, M., Zhang, Y., Huang, C., et al.: Efficient attribute-based searchable proxy re-encryption with policy hiding over IoT data[J]. *IEEE Internet of Things Journal*, (2025)
10. Orobosade, A., Charles, U., Olorunjube, F., et al.: Cyber incident response systems using machine learning and severity ranking algorithm[J]. *Int. J. Res. Innov. Appl. Sci.* **10**(8), 2098–2109 (2025)
11. Lin, X., Fu, Y., Wang, Z., et al.: A code-based anonymous authentication scheme for vehicular Ad Hoc networks[J]. *IEEE Internet of Things Journal*, (2025)
12. Reddy, K. V. N., Reddy, D. V., Babu, P. R., et al.: A Novel Proxy Re-Encryption Technique for Secure Data Sharing in Cloud Environment[C]. 2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS). IEEE, 1-5 (2024)

13. Sahai, A., Waters, B.: Fuzzy identity-based encryption[C] Annual international conference on the theory and applications of cryptographic techniques. Berlin, Heidelberg: Springer Berlin Heidelberg, 457–473 (2005)
14. Bethencourt, J., Sahai, A., Waters, B.: Ciphertext-policy attribute-based encryption[C] 2007 IEEE symposium on security and privacy (SP'07). IEEE, 321–334 (2007)
15. Li, J., Wang, Q., Wang, C., et al.: Fuzzy keyword search over encrypted data in cloud computing[C] 2010 Proceedings IEEE INFOCOM. IEEE, 2010: 1–5
16. Chaudhari, P., Das, M.L.: Privacy preserving searchable encryption with fine-grained access control[J]. *IEEE Trans. Cloud. Comput.* **9**(2), 753–762 (2019)
17. Zuo, C., Sun, S.F., Liu, J.K., et al.: Forward and backward private DSSE for range queries[J]. *IEEE Trans. Dependable Secure Comput.* **19**(1), 328–338 (2020)
18. Liu, P., He, Q., Chen, Y., et al.: Efficient multi-keyword searchable and verifiable data sharing for cloud-edge collaboration intelligent transportation systems[J]. *IEEE Internet Things J.* **12**(11), 15869 (2025)
19. Zhang, Y., Zhu, T., Guo, R., et al.: Multi-keyword searchable and verifiable attribute-based encryption over cloud data[J]. *IEEE Trans. Cloud. Comput.* **11**(1), 971–983 (2021)
20. Chen, L., Xu, S., Jin, C., et al.: Partially hidden policy attribute-based multi-keyword searchable encryption with verification and revocation[J]. *IEEE Trans. Mob. Comput.* **12**(11), 15869–15882 (2025)
21. Li, C., Chen, R., Wang, Y., et al.: REEDS: an efficient revocable end-to-end encrypted message distribution system for IoT[J]. *IEEE Trans. Dependable Secure Comput.* **21**(5), 4526–4542 (2024)
22. Ge, C., Liu, Z., Susilo, W., et al.: Attribute-based encryption with reliable outsourced decryption in cloud computing using smart contract[J]. *IEEE Trans. Dependable Secure Comput.* **21**(2), 937–948 (2023)
23. Liu, J., Li, Y., Sun, R., et al.: EMK-ABSE: efficient multikeyword attribute-based searchable encryption scheme through cloud-edge coordination[J]. *IEEE Internet Things J.* **9**(19), 18650–18662 (2022)
24. Bera, S., Prasad, S., Rao, Y.S.: Verifiable and Boolean keyword searchable attribute-based signcryption for electronic medical record storage and retrieval in cloud computing environment[J]. *J. Supercomput.* **79**(18), 20324–20382 (2023)
25. Bera, S., Rao, Y.S.: Searchable attribute-based proxy re-encryption: keyword privacy, verifiable expressive search, and outsourced decryption[J]. *SN Comput. Sci.* **5**(5), 479 (2024)
26. Fan, C.I., Wu, S.J., Tseng, Y.F., et al.: Attribute-Based Encryption Supporting Multi-Keyword Search with Effective User Revocation in Public Cloud Storage[J]. *IEEE Trans. Dependable Secure Comput.* (2025)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.